AI Workflow Documentation

Tools Used

Primary tool: Claude Code, used as a CLI/IDE-embedded agent across the full SDLC, implementation, debugging, refactoring, test generation, and documentation, not just initial code generation. Secondary: the Claude API (`claude-haiku-4-5`) integrated as a runtime dependency of the application itself (the URL safety check), so it's also a production tool, not only a development-time one.

Spec-Driven Development

The core discipline behind this project is treating AI generation as implementation of a spec, not interpretation of a request. Before any class was generated, I worked out the low-level design first: class name, dependencies, method signatures, and error conditions. That design is what got handed to the AI, not the original one-line requirement.

Concretely, `docs/architecture.md`'s control-flow diagrams and data model were written before implementation started, and each generation prompt was a structured spec pulled directly from that document: the exact package/class name, what gets injected, the method contract, and the failure modes to handle. This is the same translation the role calls out directly, turning low-level design artifacts (class structures, API contracts, data models) into structured specs that guide AI-assisted implementation, rather than freeform prompting against a vague requirement.

Example prompt, derived from the write-path spec in `architecture.md`:

> "Write `ReadService.resolveAndTrack(code, ip, referrer, userAgent)`. It should: (1) check Redis cache first, (2) on miss query PostgreSQL, (3) check expiry, (4) populate cache, (5) always publish a click event to Kafka regardless of cache hit/miss. Kafka publish failures must be caught and logged, never propagated."

Custom Instructions

There's a project-level `CLAUDE.md` at the repo root that captures conventions I don't want to restate in every prompt: no unnecessary abstractions, Lombok `@RequiredArgsConstructor` over manual constructors, no try-catch unless a failure mode is explicitly identified, and the error-handling contract (custom exception, then `GlobalExceptionHandler`, then a typed HTTP status). This is the team-level version of prompting. It's a set of instructions that makes every subsequent prompt shorter and every output more consistent, instead of re-deriving the same conventions in each request.

Agentic / CLI Workflow

Working in a CLI-driven agent, instead of a chat window with copy-paste, changes the review loop in a specific way. The agent can run the build and test suite itself and iterate against real output, so "does this compile and pass" is verified before I even look at the diff. That doesn't remove engineer review, but it does change what I'm reviewing for. Every AI-run command is visible before or as it executes, and nothing destructive (git push, force operations, deleting files outside a scratch area) runs without explicit sign-off.

Example: diagnosing a test-suite gap. While reviewing the build, the agent ran the full test suite and found that `UrlControllerIT.java`, 4 integration tests, was never actually executing. `mvn test` (Surefire) only picks up `*Test.java` by naming convention. The file was named with the `*IT.java` convention instead, which is the Failsafe plugin's convention, and there was no `maven-failsafe-plugin` bound in `pom.xml`. The tests weren't failing, they were dead code, silently skipped on every build. Root cause diagnosis, the `pom.xml` fix (binding `maven-failsafe-plugin` to `integration-test`/`verify`), and re-running to confirm all 14 tests (10 unit plus 4 integration) now pass happened in a single reviewed loop.

That surfaced a second issue: the newly-running integration tests took 52 seconds because Spring Kafka's auto-configured `KafkaAdmin` was attempting real topic-creation calls against a broker that doesn't exist in the test profile (`localhost:9093`), and the `@KafkaListener` container was retrying real broker connections on top of that. Fix: `spring.kafka.listener.auto-startup: false` plus tightened `spring.kafka.admin.properties` timeouts in `application-test.yml`, scoped to the test profile only so production config stays untouched. Verified back down to 20 seconds, same 14/14 pass count. Both fixes are two-line, low-risk, and immediately verified by re-running the exact command that surfaced the problem in the first place.

What AI Generated vs. Engineer-Edited vs. Rejected

Fully AI-generated (accepted as-is after review)

All JPA entity classes (`ShortUrl`, `ClickEvent`), all repository interfaces, all DTO classes, `GlobalExceptionHandler`, `KafkaConfig`, `WebConfig`, all custom exception classes, `ClickEventMessage`/`ClickEventProducer`/`ClickEventConsumer`, the Docker Compose file, the frontend API client (`client.ts`), frontend component structure and Tailwind styling.

AI-generated, engineer-edited

- `ReadService`: the initial draft omitted the Kafka publish call on cache hits. I added it after review.
- `ShortCodeGenerator`: the initial draft mishandled `number == 0` in `toBase62`. Added that edge case.
- `application-test.yml`: AI generated a template. I added the H2 dialect, moved the Redis test port to avoid colliding with a real local Redis, added a `:test` suffix to the counter key, and later added the Kafka test-profile timeout/auto-startup tuning described above.
- `UrlController`: AI initially called `getClientIp()` twice, passing `null` into `shorten()` on one path. Fixed to extract the IP once and reuse it for both the rate-limit check and the service call.
- `RateLimitConfig`: AI used the Bucket4j API correctly but picked `Refill.intervally` over `Refill.greedy`. Changed it for smoother rate-limiting behavior.
- `pom.xml`: the agent diagnosed the missing failsafe binding and proposed the plugin block. I checked it against the existing `spring-boot-maven-plugin` config before applying it, to make sure there wasn't a duplicate plugin declaration.

AI-generated, rejected

- A single `UrlShortenerService` "god class" holding all business logic. Rejected in favor of the `WriteService`/`ReadService`/`AnalyticsService`/`UrlInfoService` split, which separates read and write concerns and is independently testable.
- A `ShortUrlMapper` interface built on MapStruct. Rejected as unnecessary complexity for a prototype with simple field-by-field mappings, replaced with private `toResponse()` methods.
- A custom `RedisConfig` with a hand-built `RedisTemplate`. Rejected because Spring Boot's auto-configured `StringRedisTemplate` already does what's needed for string key-value storage.

Engineer-written (not AI-generated)

The 302-vs-301 redirect decision, the `0 0 2 * * *` cleanup cron expression (verified manually against cron syntax), the choice to use `short_code` as the Kafka message key, and the integration test harness setup (`@SpringBootTest`, `@AutoConfigureMockMvc`, `@ActiveProfiles("test")`).

Quality Gates Applied

1. **Compilation/style.** Import correctness, no wildcard imports outside test utilities, Lombok annotations matched to constructor style, non-final `@Value` fields.
2. **Validation review.** `ShortenRequest` annotations (`@NotBlank`, `@URL`, `@Size`, `@Pattern`) checked against edge cases: empty optional field, spaces, hyphens, underscores.
3. **Security review.** Parameterized queries via Spring Data JPA (no raw SQL), the `@URL` validator blocking `javascript:` and other protocol injection, `X-Forwarded-For` parsed defensively (split on comma, trimmed) for proxy chains, per-IP rate limiting on the creation endpoint.
4. **Test coverage review.** 10 unit tests (generator, write, read) plus 4 integration tests (create/validate/conflict/list), all executing in a `mvn verify` run.
5. **Dependency/security scanning.** `npm audit` on the frontend surfaced 4 known CVEs: a moderate esbuild dev-server-only exposure (affects `vite <=6.4.2` transitively, fix requires a Vite 5 to 8 major bump) and a moderate React Router open-redirect/SSR-hydration issue (fix requires a 6 to 7 major bump). Both fixes are only available via `--force`, meaning breaking. I decided to defer rather than force-upgrade blind. The esbuild issue only affects the local dev server, not the production build, and the React Router bump is a real API migration that needs route-by-route testing before I'd trust it. Logged in `RISKS_AND_TRADEOFFS.md` as an accepted, documented risk with a visible follow-up rather than silently run and hope nothing broke.

Traceability Table

Task	Prompt Type	Outcome	Decision
`ShortUrl` entity	"Generate JPA entity with these fields"	Generated, accepted	No change
`ShortCodeGenerator`	"Counter-based Base62 with Redis batch"	Generated, edited	Fixed `toBase62(0)` edge case
`WriteService`	"Shorten + delete with cache"	Generated, accepted	Reviewed cache key format
`ReadService`	"Cache-first with Kafka publish"	Generated, edited	Added publish on cache-hit path
`ClickEventProducer`	"Fire-and-forget with try-catch"	Generated, accepted	Verified exception scope
`RateLimitConfig`	"Per-IP Bucket4j token bucket"	Generated, edited	Changed `Refill` type
God-class service	"All logic in one service"	Generated, rejected	Split into 4 services
MapStruct mapper	"DTO mapping via MapStruct"	Generated, rejected	Replaced with inline `toResponse()`
`RedisConfig`	"Custom RedisTemplate bean"	Generated, rejected	Spring auto-config was sufficient
Unit tests	"Mockito tests for X"	Generated, accepted	Verified assertions
Integration tests	"MockMvc tests for controller"	Generated, edited	Added `@ActiveProfiles`
`UrlSafetyService`	"Call Claude API, return safe/reason JSON"	Generated, edited	Fixed prompt bias (example showed `safe:true`), added markdown code-fence stripping
Failsafe wiring	"Why do the 4 integration tests never show in output?"	Diagnosed, fix generated	Bound `maven-failsafe-plugin`, verified 14/14 pass

Task	Prompt Type	Outcome	Decision
Kafka test timeout	"Why does verify take 52s?"	Diagnosed, fix generated	Disabled listener auto-start, tightened admin timeouts in test profile, verified 20s with the same pass count

Human Sign-Off Points

These decisions required human judgment and weren't delegated to AI:

1. Read/write service split, based on the 100:1 read:write ratio assumption.
2. 302 vs. 301, chosen to preserve analytics completeness and support deletion/expiry.
3. Redis counter batch size (1000), balancing Redis call frequency against ID waste on restart.
4. Rate-limit scope: creation only, not redirects.
5. Accepting analytics loss on Kafka failure: redirect availability matters more than analytics completeness.
6. In-memory rate-limit storage, accepted for the prototype and flagged as a multi-instance gap.
7. No authentication, accepted as a scope limitation and flagged as the top production priority.
8. Fail-open URL safety check: a failed AI check must never break the core shortening function.
9. Deferring the `npm audit --force` fixes rather than applying them unreviewed.

Lessons Learned

1. **Specification quality determines output quality.** "Make it enterprise-ready" produces a 200-line god class. A named interface with explicit error conditions produces usable code on the first pass.
2. **Review structurally, not just syntactically.** The god-class issue compiled fine and wouldn't have tripped a linter. It's a design review finding, not a syntax one.
3. **AI doesn't know your implicit conventions unless you state them per prompt, or capture them once in a shared instructions file** (see `CLAUDE.md`). The Redis key format `url:{code}` stayed consistent across three separately-generated classes only because I specified it each time. A shared conventions file is the team-scale version of that fix.
4. **Test generation is a second design pass, not just validation.** Writing the cache-hit test for `ReadService` is what surfaced the missing Kafka publish call. The test failed to find something to assert on before it failed to pass.
5. **Reject over-engineering early and say why.** The MapStruct and custom `RedisConfig` rejections each took under a minute to catch, and having the rationale written down is what makes the rejection defensible in review rather than just a preference.
6. **Agent-run verification changes what review is for.** When the agent can build and test its own output before I look at it, my review time shifts from "does this compile" to "is this the right design, and did the agent actually understand the failure it just fixed." That's a better use of the time either way.
7. **A passing build isn't proof of coverage.** The dead-integration-test case above is the clearest example: `mvn test` reported success while running 10 of the 14 tests the project actually had. Worth checking what a test run actually executed, not just whether it exited green.

Team-Level AI-Adoption Notes

If this pattern were rolled out beyond a single engineer, the `CLAUDE.md` custom-instructions file and the traceability table format above are the two artifacts worth standardizing across a team. They're what make AI-assisted output auditable after the fact instead of just fast to produce. The traceability table in particular is cheap to maintain incrementally, one row per non-trivial generation, and expensive to reconstruct after the fact, so it should be a normal PR-description habit rather than a one-time assessment artifact.